SIDInspector: A Mapping-First Diagnostic Resource for Semantic-ID Tokenizers

Jiandong Ding, Heng Chang, Huijie Qin, and Tianying Liu
{dingjiandong2,heng.chang,qinhuijie,liutianying2}@huawei.com
Huawei Technologies
Shanghai, China

Abstract

Semantic-ID (SID) tokenizers are increasingly reused as standalone artifacts in generative recommendation: an exported item-to-code mapping becomes the address space that a later sequence generator must use. Yet these mappings rarely come with a common inspection interface. Before training a generator, a user lacks a single way to check whether the catalog is covered, whether full codes alias many items, whether prefixes reflect user co-occurrence, whether tail items receive capacity, or whether the prefix structure creates large fan-out. We present SIDInspector, a mapping-first diagnostic resource for SID tokenizer artifacts. SIDInspector defines a small adapter contract over item mappings, metadata, interactions, and optional generator traces; validates the contract; and reports mapping-level probes for utilization, aliasing, neighborhood alignment, popularity allocation, and structural cost, with hooks for temporal churn and generator traces.

The value of SIDInspector is not a leaderboard but an inspectable artifact profile. On a 23,742-item Musical worked example, the same adapter contract shows that a controlled GRID/RQ-KMeans-style feature-text export has 3,749 unique full codes and a 0.977 full-code aliasing rate, while a bounded ReSID/GAOQ export is aliasing-free in its exported mapping. More importantly, the strongest prefix–co-occurrence alignment comes from a deterministic category-prefix control, not from either learned export row (0.447 versus 0.154 and 0.055–0.080). This counterintuitive result shows why addressability and behaviorally meaningful prefixes should be audited separately. Auxiliary checks replicate the diagnostic direction on All_Beauty, add a real Sports GRID export with complete joins, and show that D3 is positively associated with candidate recall under a fixed train-only reranker. Mechanism probes further separate raw aliasing volume, interaction-qualified aliasing risk, head-to-tail capacity, and realized prefix cost. SIDInspector is not a new tokenizer or a benchmark suite; it turns SID tokenizer exports into inspectable resource artifacts before downstream generator training.

CCS Concepts

• **Information systems** → **Recommender systems**; *Retrieval models and ranking*.

Keywords

semantic IDs, generative recommendation, tokenizer artifacts, diagnostic evaluation, recommender systems

ACM Reference Format:
Jiandong Ding, Heng Chang, Huijie Qin, and Tianying Liu. 2026. SIDInspector: A Mapping-First Diagnostic Resource for Semantic-ID Tokenizers. In *CIKM '26, November 7–11, 2026, Rome, Italy*. ACM, New York, NY, USA, 5 pages.

CIKM '26, Rome, Italy
2026.

1 Introduction

Semantic-ID (SID) tokenization has become a practical artifact boundary for generative recommendation. Systems such as TIGER map items into discrete code sequences so that a sequence model can generate candidate identifiers rather than score every catalog item directly [21]. Once exported, that item-to-code mapping becomes the address space exposed to the generator. Recent work has expanded this space quickly: residual quantization frameworks, recommender-native encoding, collaborative tokenization, differentiable tokenization, non-uniform capacity, qualified collisions, variable-length interfaces, and drift-aware refreshes all change the artifact that downstream users must inspect before generator training [3, 6–8, 11, 15, 24, 28]. This paper treats those exports as a shared artifact interface: the goal is not to reimplement the literature, but to make item-to-code mappings inspectable under the same contract.

The reusable artifact interface has not caught up with this boundary. A new tokenizer repository may expose checkpoints, item mappings, codebooks, or only intermediate feature files. Downstream users then need to answer basic engineering questions before training a generator: do all catalog items have valid codes, are many items aliased to the same full code, do prefixes reflect user co-occurrence rather than only metadata taxonomy, are tail items allocated enough address capacity, and does the prefix structure create large fan-out? Today these checks are usually reimplemented ad hoc inside each paper. The resulting evidence is difficult to compare because each method reports a different mixture of codebook usage, downstream ranking, and qualitative examples.

Aggregate ranking metrics remain necessary, but they are the wrong interface for inspecting the exported address space itself. Prior recommender tooling has argued for behavioral tests beyond NDCG-style aggregates [4]; SID tokenizers add a more concrete resource problem: the item mapping should be validated and profiled before a full generator consumes GPU time. Underused codebooks, repeated full codes, behaviorally weak prefixes, tail compression, and large prefix fan-out can be hidden by a downstream score, yet they are direct properties of the reusable tokenizer artifact.

We present SIDInspector, a diagnostic/interface resource for inspecting item-to-SID tokenizer artifacts before or alongside downstream recommendation evaluation. SIDInspector is deliberately neither a new tokenizer nor a RecBole-style coverage benchmark. It asks a narrower and reusable question: given any tokenizer artifact that can emit item-level code sequences, what can be said about its capacity use, aliasing, behavioral alignment, head-tail allocation, and structural cost before retraining a full generator?

The resource contribution is threefold. First, SIDInspector defines a small adapter contract for SID assignments, item metadata, interaction histories, and optional generator outputs, plus validators that reject incomplete or ambiguous artifacts before metrics are reported. Second, it implements D1–D5 mapping-level diagnostic probes for utilization, aliasing, neighborhood alignment, popularity allocation, and structural cost, with D6 churn support for continual-tokenizer settings. Third, it provides worked examples and controlled mechanism probes that make the probes interpretable rather than just available. Two public SID export paths show how adapters become comparable artifact-profile rows; controlled probes separate full-code aliasing pressure, interaction-qualified aliasing risk, collaborative-prefix recovery, tail capacity, and structural prefix fan-out. The resulting diagnostic finding is simple but important: in the bounded Musical worked example, a category-prefix control recovers more co-occurrence prefix neighbors than the learned export rows. Learned addressability and behavioral prefix alignment therefore require separate inspection. Auxiliary vertical, fixed-reranker, and portability checks defend this finding without turning the paper into a method leaderboard [5].

2 Artifact Interface and Validation

SIDInspector treats a tokenizer export as an inspectable resource artifact, not as an opaque component of a trained recommender. The minimum input is a table `sid_assignments(item_id, sid_0, ..., sid_L)`. Optional tables provide metadata, interactions, refresh pairs, and generator outputs. This mapping-first contract lets adapters normalize heterogeneous repositories while keeping the audit independent of a particular training loop. Figure 1 summarizes the workflow and previews the diagnostic signals that the resource is designed to expose.

Artifact contract. The required contract has three checks: each row has a stable item key, every SID level is a discrete token, and the diagnostic item universe is joinable to metadata and interaction slices. Metadata enables semantic/category slices; interactions enable collaborative-neighborhood and head-tail diagnostics. The `generator_outputs` table is optional because many public tokenizer releases expose item codes but no trained generator traces.

Adapter specification. An adapter is a thin export layer, not a retraining recipe. It must emit `sid_assignments` with stable item keys, a method/dataset label, and one column per SID level. If available, it also emits metadata, interactions, refresh-pair snapshots, or generator traces using the same item keys. The validator checks key uniqueness, missing mappings, depth consistency, join coverage, and declared evidence role before any row can enter the audit tables. The minimal schema is deliberately small:

$$\texttt{item\_id}, \texttt{method}, \texttt{dataset}, \texttt{sid\_0}, \ldots, \texttt{sid\_L} \rightarrow \{\texttt{D1}, \ldots, \texttt{D5}\}.$$

Interactions add D3/D4 slices; paired mappings add D6; generator traces add D7. This makes a new tokenizer easy to attach without rewriting D1–D5. Current D1–D5 reports use a rectangular code table; variable-length tokenizers can be attached by exporting padded or masked levels plus realized length, but variable-length serving behavior remains structural D5 evidence unless generator traces are available.

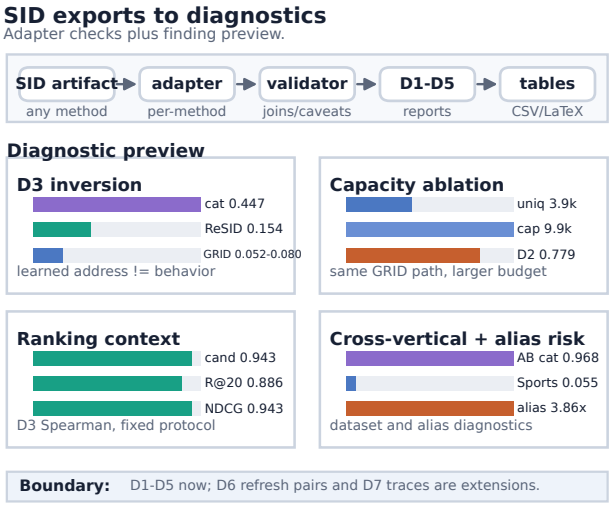

Figure 1: SIDInspector workflow and finding preview. Top: adapter and validator normalize a SID artifact before D1–D5 reports. Bottom: preview panels show the D3 inversion, capacity ablation, ranking context, and portability/aliasing checks detailed in Tables 2 and 3.

Terminology. A *diagnostic probe* is a named metric family: D1 utilization, D2 aliasing, D3 neighborhood alignment, D4 popularity allocation, D5 structural cost, D6 temporal churn, or D7 generation traces. A *controlled mechanism probe* is an input row designed to activate one known mechanism and check whether the diagnostic probe reacts. Current paper evidence uses D1–D5. D6 is implemented as an extension for paired refresh mappings and has one repository smoke result; D7 is only an interface hook until generator outputs or beam traces are supplied.

Claim scope. SIDInspector separates rows by what they can support. A named-method row comes from a public method path and can support named-method artifact claims. A controlled mechanism probe is deliberately synthetic or method-inspired and tests whether a diagnostic reacts to a known mechanism, but it is not named-method coverage. Optional rows exercise extensions such as drift or non-Amazon schemas. Literature-only rows motivate the design space but are not loaded as runnable evidence.

Validation before metrics. Adapters are refused as evidence until they pass item-count agreement, missing-code, duplicate-key, depth-consistency, and provenance checks. This guard is necessary because public SID repositories may expose checkpoints, item mappings, codebooks, intermediate feature files, or only partial releases. The manifest records each row's evidence role and caveats before any diagnostic number is emitted.

Prior recommender resources such as RecList and Elliot motivate diagnostic and reproducible evaluation beyond a single aggregate metric [1, 4]. SIDInspector applies that resource logic to a newer artifact boundary: the item address space consumed by generative recommenders.

Table 1: Evidence catalog for the review artifact.

Row	Type	Items	Seeds	D coverage
GRID ft	named (A)	23,742	3	D1–D5
GRID ft-cap	named (A, capacity ablation)	23,742	1	D1–D5
RQ-min ref	reference adapter	23,742	1	D1–D5
ReSID†	named (B)	23,742	1	D1–D5
Cat-prefix, Pop-bal, Hash-coll	controls	23,742	n/a	D1–D5
Mechanism probes (3)	controlled	synth.	n/a	D2,D4,D5
All_Beauty, Sports, MovieLens, DACT	portability/extension	varies	varies	D1–D5, D6

3 Diagnostic Probes and Evidence Roles

SID papers now optimize different artifact properties: learned item indexing and TIGER-style generative retrieval established the interface [9, 21], while later work stresses collaborative alignment, end-to-end or differentiable tokenization, representation-source sensitivity, code-assignment diversity, semantic-collaborative disentanglement, collision qualification, variable-length or asymmetric interfaces, drift, and deployment surfaces [2, 3, 7, 8, 10, 12, 16–18, 20, 23, 27, 28]. The literature space is broader than the artifacts that SIDInspector can honestly execute. Table 1 summarizes what the review artifact ships rather than counting baselines. The taxonomy guides future adapters; it is not presented as reproduced-method coverage.

D1–D5 mapping-level probes. D1, *utilization*, reports per-level usage, prefix counts, imbalance summaries, and dead or underused code indicators. D2, *aliasing*, measures duplicate full SIDs and duplicate prefixes; it is an aliasing profile, not a causal harm estimate. A bounded interaction-qualified mechanism probe is included because collision-aware work argues that duplicate-code assignments are not uniformly harmful [8]. D3, *neighborhood alignment*, asks whether prefix neighborhoods recover item co-occurrence neighbors, keeping metadata purity as auxiliary context rather than as collaborative quality. D4, *popularity allocation*, splits items by popularity and measures whether full-code capacity and prefix structure are allocated differently across head, mid, and tail items. D5, *structural cost*, reports SID length, prefix fan-out, duplicate codes, and trie-like expansion pressure. D5 is not serving latency; long, variable-length, or asymmetric SID methods make that distinction important [3, 10, 25, 26].

Extension probes. D6, *temporal churn*, computes mapping changes between tokenizer refreshes for drift-aware settings [2, 6]. It is implemented as optional refresh-pair evidence, not a required main-paper claim. D7, *generation traces*, covers invalid paths, next-token entropy, and duplicate generated candidates. D7 requires generator-output tables or beam traces and is deliberately not implemented as current evidence.

4 Worked Examples and Probe Evidence

SIDInspector is evaluated as an interface resource, not as a leaderboard over tokenizer systems. The worked example asks whether one adapter contract can turn heterogeneous SID exports into comparable artifact profiles. GRID supplies a released GRID/RQ-KMeans-style residual-quantization module; ReSID supplies processed Musical artifacts and a bounded GAOQ export path. Sanity rows and controlled mechanism probes then calibrate which diagnostic differences are structural, behavioral, or merely artifacts of capacity.

Table 2: SID diagnostics on Amazon Reviews 2023 Musical Instruments. Rows follow the diagnostic path: RQ-style variants, ReSID contrast, then controls. Bold rows are named-tokenizer export paths; italics are controls.

Artifact	Unique SIDs	D2	D3	D4	D5
GRID ft	3,749	0.977	0.055	0.370	64/3.4k/3.7k
GRID ft-cap	9,874	0.779	0.080	0.639	32/9300/9874
RQ-min ref	17,247	0.440	0.065	0.883	32/2368/17.2k
ReSID†	23,742	0.000	0.154	1.000	32/1280/23.7k
Cat-prefix†	23,742	0.000	0.447	1.000	30/83/313/23.7k
Pop-balanced	22,707	0.086	0.303	0.962	4/1024/22.7k
Hash-collide	256	1.000	0.004	0.032	256/256/256/256

Table 3: Controlled mechanism probes. Each compact contrast activates a known artifact pressure and checks whether the targeted diagnostic separates it.

Probe	D	Without probe	With probe
Qualified aliasing	D2	hash 1.19x	co-occur 3.86x
Capacity budget	D1/D4	head 1.000	tail 0.028
Variable depth	D5	max-depth 12,010	active 7,914

Worked-example protocol. The paper-facing case study uses the same 23,742 Musical items for both named export rows. The GRID row is a controlled feature-text export: ReSID processed feature text is embedded and passed through a released GRID/RQ-KMeans-style residual k-means module. The ReSID row is a bounded FAMAE-to-GAOQ export on the same item universe. This modest protocol gives reviewers a same-item diagnostic contrast while avoiding the stronger, unsupported claim that the paper reproduces every training choice of TIGER, GRID, or the full ReSID pipeline.

Table 2 shows how two exports become comparable artifact-profile rows under one schema. Its order is intentional: the RQ-style rows are adjacent so that capacity and aliasing changes can be read before the ReSID contrast and the sanity controls. The GRID ft row is intentionally labeled as feature-text: it uses processed feature text from the ReSID artifact path and the released residual k-means module, not a faithful raw-text TIGER reproduction. Under that controlled setup, the row exposes high aliasing pressure and limited tail capacity. The ReSID/GAOQ row exports one full code per item in this bounded run, yielding zero full-code collisions and higher D3-L1 collaborative prefix recovery. The daggered rows make the structural floor explicit: collision-free full codes are not by themselves evidence of better recommendation quality. The capacity-matched GRID row reopens the strongest alternative explanation: when the same feature-text path uses a larger 32/1280/1280 budget, unique full codes rise to 9,874 and D2 aliasing drops to 0.779, but the artifact still does not approach the structurally item-unique ReSID or category-prefix rows. Thus capacity explains part of the original GRID pressure. The ablation is not a full item-unique leaf match; additional capacity could reduce aliasing further, so the contrast remains an artifact profile rather than a method ranking. The RQ-min row is included for a different reason: it is a minimal

residual-quantization reference adapter that passes the same validator on the full Musical item universe. Its role is to show that SIDInspector accepts an independent SID-generation code path; it is not third named-method coverage and it does not reproduce TIGER, GRID, or ReSID.

Diagnostic findings. The worked example supports one central finding and two supporting ones. The central finding is that addressability is not behavioral prefix alignment. The ReSID row is structurally item-unique; the category-prefix control is also alias-free because it uses metadata categories as early levels and a within-prefix item index as the leaf. It is a taxonomy control rather than a learned tokenizer. D3 exposes the surprising part of the profile: the non-learned category-prefix row recovers more level-1 co-occurrence neighbors (0.447) than ReSID (0.154) or either GRID row (0.055–0.080). The right interpretation is not that category IDs are better tokenizers. It is that learned or item-unique addresses do not guarantee behaviorally aligned prefixes. This failure mode is visible only when the mapping is inspected directly.

The first supporting finding is intervention-style: a capacity change fixes one failure mode without repairing another. Increasing the GRID feature-text budget to 32/1280/1280 raises unique full codes from the 3.7–4.0k range to 9,874 and reduces D2 aliasing from 0.977 to 0.779. That ablation weakens a simple "too little capacity" objection, but it does not make the row item-unique. The RQ-style rows expose a sharper diagnostic dissociation: D2 aliasing falls from 0.977 (GRID ft) to 0.779 (GRID ft-cap) and 0.440 (RQ-min), while D3 stays in the narrow 0.055–0.080 range. Capacity and aliasing can improve without producing behaviorally aligned prefixes, which is why these probes are reported separately. Table 3 gives the same point as controlled mechanism checks: qualified aliasing separates co-occurrence aliases from hash aliases, capacity probes separate head and tail allocation, and variable-depth probes separate maximum-depth from realized prefix cost.

We also test whether the central D3 finding is fragile. It is not confined to the Musical table: on an All_Beauty 20k vertical, a coarse category-prefix control reaches D3-L1 0.968 while GRID/RQ-KMeans feature-text rows are 0.081, 0.087, and 0.090 across seeds 42–44. The coarse-category flag is essential–this is diagnostic portability evidence, not a claim that taxonomy IDs are better tokenizers. The unusually high All_Beauty value is itself part of the diagnostic signal: D3 distinguishes a vertical where the available coarse taxonomy is strongly aligned with observed co-occurrence from Musical, where the same category-prefix control is only moderately aligned. A plausible explanation is that Beauty co-purchase sessions often stay within narrow product-type and routine categories, while Musical accessory and instrument co-purchases can cross coarser taxonomy boundaries. To keep D3 from being only an internal score, we also report bounded ranking context: a 5,000-user Musical check uses SID prefixes only as candidate sets and applies the same train-only co-occurrence/popularity reranker to all six artifact/control rows; D3 correlates with candidate recall (Spearman 0.943) and fixed-reranker Recall@20 (0.886). A bounded All_Beauty temporal-LOO replication is consistent in direction across four rows: D3 is monotonic with candidate recall and fixed-reranker Recall@20, but the small row count and constructed split keep it as supplementary directional evidence. Finally, the GRID export path is not confined to one vertical: a real Sports 20k GRID export has complete metadata and interaction joins, D3-L1 0.055, duplicate-SID rate 0.592, and D5 prefixes 128/7986/8165. Sports proxy rows remain supplement-only and do not add named method coverage. DACT D6 and MovieLens schema rows exercise extension and portability paths, but the main paper claim remains the Musical worked example.

5 Artifact Availability and Limits

The artifact repository contains the SIDInspector Python package, adapter scripts, metric runners, generated CSV/Markdown/LaTeX tables, provenance logs, BibTeX/source-audit notes, an MIT license, and a reviewer quickstart. The reviewer entry point is the tag `audit-sid-cikm-resource-v0.1`, with a clean-checkout verifier for the published tables and figure; the anonymous review artifact is available at https://anonymous.4open.science/r/sidinspector-9BB2. Large local artifacts are kept out of git and described through manifests so reviewers can separate public reproducibility assets from local caches.

The resource is intended to be used in three modes. A downstream recommender researcher can use the diagnostics as a pre-training triage step: artifacts with missing items, inconsistent depth, severe aliasing, or weak collaborative-prefix recovery can be flagged for inspection before generator training consumes GPU time. A method author can start from the minimal adapter template, emit the normalized SID-assignment table, then receive D1–D5 reports without changing their training loop. The artifact also includes runtime notes: D1/D2/D4/D5 are mapping-level preflight checks, while D3 is the dominant interaction-neighborhood step. A paper reviewer can run the clean-check verifier and compare published CSVs with the PDF tables. These signals do not replace Recall@K or NDCG; they make tokenizer artifacts inspectable before a full benchmark or generator-training run.

The evidence supports a resource-interface claim, not broad method coverage. The GRID Musical row is a controlled feature-text export rather than an end-to-end TIGER or GRID reproduction; ReSID is the bounded Musical export; RQ-min is a local reference adapter; the Sports balanced GAOQ path was stopped after constrained k-means became CPU-bound; and CARD remains a fallback/probe path rather than faithful CARD reproduction [24]. D2 is not causal collision harm; D3 uses prefix-neighborhood and reranker checks, not trained-generator ranking; D5 is structural cost rather than measured latency; D6 is optional churn evidence; and D7 requires generator traces absent from the current rows.

New adapters emit the same normalized tables, declare their evidence role, and record feature or checkpoint provenance. A named tokenizer improves coverage only when item-level codes pass the validator and join to metadata or interactions; a controlled mechanism probe improves diagnostic calibration but does not count as named-method coverage; a generator trace activates D7. Future versions should add causal collision-harm validation, broader faithful adapter coverage, generator-output diagnostics, and unified search/recommendation checks [14, 19]. Industrial SID deployment and ranking studies further motivate treating these artifact properties as engineering objects [12, 13, 22].

GenAI Usage Disclosure

Generative AI tools assisted with language editing and code debugging. All experimental results were produced by author-written scripts and verified against saved artifacts.

References

[1] Vito Walter Anelli, Alejandro Bellogin, Antonio Ferrara, Daniele Malitesta, Felice Antonio Merra, Claudio Pomo, Francesco M. Donini, and Tommaso Di Noia. 2021. Elliot: A Comprehensive and Rigorous Framework for Reproducible Recommender Systems Evaluation. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21)*. Association for Computing Machinery, New York, NY, USA, 2405–2414. doi:10.1145/3404835.3463245

[2] Vladimir Baikalov, Iskander Bagautdinov, and Sergey Muravyov. 2026. Mitigating Collaborative Semantic ID Staleness in Generative Retrieval. arXiv:2604.13273 [cs.IR] Accepted by SIGIR 2026.

[3] Wenzhuo Cheng, Menghang Gong, Qixin Guo, Hang Zheng, Zhaobin Yang, Jianguo Lou, and Zhengwei Zheng. 2026. CapsID: Soft-Routed Variable-Length Semantic IDs for Generative Recommendation. arXiv:2605.05096 [cs.IR]

[4] Patrick John Chia, Jacopo Tagliabue, Federico Bianchi, Chloe He, and Brian Ko. 2022. Beyond NDCG: Behavioral Testing of Recommender Systems with RecList. In *Companion Proceedings of the ACM Web Conference 2022 (WWW '22 Companion)*. Association for Computing Machinery, New York, NY, USA, 99–104. doi:10.1145/3487553.3524215

[5] CIKM 2026 Organizing Committee. 2026. CIKM 2026 Resource Papers. https://cikm2026.diag.uniroma1.it/resource-papers/. Accessed 2026-05-19.

[6] Yuebo Feng, Jiahao Liu, Mingzhe Han, Dongsheng Li, Hansu Gu, Peng Zhang, Tun Lu, and Ning Gu. 2026. Drift-Aware Continual Tokenization for Generative Recommendation. arXiv:2603.29705 [cs.IR]

[7] Junchen Fu, Xuri Ge, Alexandros Karatzoglou, Ioannis Arapakis, Suzan Verberne, Joemon M. Jose, and Zhaochun Ren. 2026. Differentiable Semantic ID for Generative Recommendation. arXiv:2601.19711 [cs.IR] Accepted by SIGIR 2026.

[8] Zheng Hu, Yuxin Chen, Yongsen Pan, Xu Yuan, Yuting Yin, Daoyuan Wang, Boyang Xia, Zefei Luo, Hongyang Wang, Songhao Ni, Dongxu Liang, Jun Wang, Shimin Cai, Tao Zhou, Fuji Ren, and Wenwu Ou. 2026. Stop Treating Collisions Equally: Qualification-Aware Semantic ID Learning for Recommendation at Industrial Scale. arXiv:2603.00632 [cs.IR]

[9] Wenyue Hua, Shuyuan Xu, Yingqiang Ge, and Yongfeng Zhang. 2023. How to Index Item IDs for Recommendation Foundation Models. In *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region (SIGIR-AP '23)*. Association for Computing Machinery, New York, NY, USA, 195–204. doi:10.1145/3624918.3625339

[10] Bin Huang, Xin Wang, Junwei Pan, Yongqi Zhou, Yifeng Zhou, Zhixiang Feng, Shudong Huang, Haijie Gu, and Wenwu Zhu. 2026. Asymmetric Generative Recommendation via Multi-Expert Projection and Multi-Faceted Hierarchical Quantization. arXiv:2605.14512 [cs.IR]

[11] Clark Mingxuan Ju, Liam Collins, Leonardo Neves, Bhuvesh Kumar, Louis Yufeng Wang, Tong Zhao, and Neil Shah. 2025. Generative Recommendation with Semantic IDs: A Practitioner's Handbook. arXiv:2507.22224 [cs.IR]

[12] Clark Mingxuan Ju, Tong Zhao, Leonardo Neves, Liam Collins, Bhuvesh Kumar, Jiwen Ren, Lili Zhang, Wenfeng Zhuo, Vincent Zhang, Xiao Bai, Jinchao Li, Karthik Iyer, Zihao Fan, Yilun Xu, Yiwen Chen, Peicheng Yu, Manish Malik, and Neil Shah. 2026. Semantic IDs for Recommender Systems at Snapchat: Use Cases, Technical Challenges, and Design Choices. arXiv:2604.03949 [cs.IR] Accepted to the SIGIR 2026 Industry Track.

[13] Guowen Li, Yuepeng Zhang, Shunyu Zhang, Yi Zhang, Xiaoze Jiang, Yi Wang, and Jingwei Zhuo. 2026. SID-Coord: Coordinating Semantic IDs for ID-based Ranking in Short-Video Search. arXiv:2604.10471 [cs.IR] Accepted by SIGIR 2026.

[14] Yongqi Li, Xinyu Lin, Wenjie Wang, Fuli Feng, Liang Pang, Wenjie Li, Liqiang Nie, Xiangnan He, and Tat-Seng Chua. 2024. A Survey of Generative Search and Recommendation in the Era of Large Language Models. arXiv:2404.16924 [cs.IR]

[15] Yu Liang, Zhongjin Zhang, Yuxuan Zhu, Kerui Zhang, Zhiluohan Guo, Wenhang Zhou, Zonqi Yang, Kangle Wu, Yabo Ni, Anxiang Zeng, Cong Fu, Jianxin Wang, and Jiazhi Xia. 2026. Rethinking Generative Recommender Tokenizer: Recsys-Native Encoding and Semantic Quantization Beyond LLMs. arXiv:2602.02338 [cs.IR]

[16] Chang Liu, Yimeng Bai, Xiaoyan Zhao, Yang Zhang, Fuli Feng, and Wenge Rong. 2025. DiscRec: Disentangled Semantic-Collaborative Modeling for Generative Recommendation. arXiv:2506.15576 [cs.IR]

[17] Enze Liu, Bowen Zheng, Cheng Ling, Lantao Hu, Han Li, and Wayne Xin Zhao. 2024. Generative Recommender with End-to-End Learnable Item Tokenization. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25)*. Association for Computing Machinery, New York, NY, USA, 729–739. doi:10.1145/3726302.3729989

[18] Yongsen Pan, Yuxin Chen, Zheng Hu, Xu Yuan, Daoyuan Wang, Yuting Yin, Songhao Ni, Hongyang Wang, Jun Wang, Fuji Ren, and Wenwu Ou. 2026. Beyond Static Collision Handling: Adaptive Semantic ID Learning for Multimodal Recommendation at Industrial Scale. arXiv:2604.23522 [cs.IR]

[19] Gustavo Penha, Edoardo D'Amico, Marco De Nadai, Enrico Palumbo, Alexandre Tamborrino, Ali Vardasbi, Max Lefarov, Shawn Lin, Timothy Heath, Francesco Fabbri, and Hugues Bouchard. 2025. Semantic IDs for Joint Generative Search and Recommendation. arXiv:2508.10478 [cs.IR] Accepted by RecSys 2025 Late-Breaking Results track.

[20] Shutong Qiao, Wei Yuan, Tong Chen, Xiangyu Zhao, Quoc Viet Hung Nguyen, and Hongzhi Yin. 2026. When Text-as-Vision Meets Semantic IDs in Generative Recommendation: An Empirical Study. arXiv:2601.14697 [cs.IR]

[21] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. 2023. Recommender Systems with Generative Retrieval. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 10299–10315. https://proceedings.neurips.cc/paper_files/paper/2023/hash/20dcab0f14046a5c6b02b61da9f13229-Abstract-Conference.html

[22] Anima Singh, Trung Vu, Nikhil Mehta, Raghunandan Keshavan, Maheswaran Sathiamoorthy, Yilin Zheng, Lichan Hong, Lukasz Heldt, Li Wei, Devansh Tandon, Ed H. Chi, and Xinyang Yi. 2023. Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations. arXiv:2306.08121 [cs.IR]

[23] Wenjie Wang, Honghui Bao, Xinyu Lin, Jizhi Zhang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Learnable Item Tokenization for Generative Recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24)*. Association for Computing Machinery, New York, NY, USA, 2400–2409. doi:10.1145/3627673.3679569

[24] Yibiao Wei, Jie Zou, Pengfei Zhang, Xiao Ao, Weikang Guo, Zeyu Ma, and Yang Yang. 2026. CARD: Non-Uniform Quantization of Visual Semantic Unit for Generative Recommendation. arXiv:2604.26427 [cs.IR]

[25] Ming Xia, Zhiqin Zhou, Guoxin Ma, and Dongmin Huang. 2026. Unleash the Potential of Long Semantic IDs for Generative Recommendation. arXiv:2602.13573 [cs.IR]

[26] Aoran Zhang, Yu-Bin Yang, and Yonghong Yu. 2026. Hyperbolic RQ-VAE enhanced Generative Recommendation with Differential-Length Codebook Strategy. ICML 2026. https://icml.cc/virtual/2026/poster/65614 Official ICML 2026 poster record.

[27] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2023. Adapting Large Language Models by Integrating Collaborative Semantics for Recommendation. arXiv:2311.09049 [cs.IR]

[28] Jieming Zhu, Mengqun Jin, Qijiong Liu, Zexuan Qiu, Zhenhua Dong, and Xiu Li. 2024. CoST: Contrastive Quantization based Semantic Tokenization for Generative Recommendation. arXiv:2404.14774 [cs.IR]